

GUÍA DE ESTÁNDARES DE CODIFICACIÓN Y ENTREGABLES PARA DESARROLLO

“La calidad no se prueba al final, se construye desde el código.”

Quality Assurance (QA)

Versión: 1.8

Fecha de Emisión: 03 de Marzo de 2026

ÍNDICE

1. INTRODUCCIÓN Y ALINEACIÓN **2**

2. REFERENCIAS Y RESPALDO TÉCNICO **2**

2.1 Glosario de Conceptos y Definiciones Operativas 3

3. REGLAS DE CODIFICACIÓN Y COMPLEJIDAD (HARD RULES) **8**

3.1. Complejidad de Bucles y Control de Flujo 10

3.2. Complejidad Ciclomática (Regla Explícita) 11

4. PROTOCOLO DE COMENTARIOS (DOCSTRING) **12**

5. ESTÁNDARES DE LENGUAJE Y DOCUMENTACIÓN DE PROYECTO **14**

5.1 Estándar de Lenguaje para Bases de Datos 15

6. SEGURIDAD DE APLICACIONES (OWASP NIVEL 2) **15**

6.1 Principios Generales de Seguridad (Shift-Left Security) 15

6.2 Controles OWASP Nivel 2 15

6.3 Gestión de Variables de Entorno 16

7. ADAPTACIÓN EXPLÍCITA A PRODUCTOS WEB **18**

7.1. Reglas Específicas para Backend / API 18

7.1.1 Consistencia de Idioma en Endpoints (URLs) 18

7.2. Reglas Específicas para Frontend 18

7.3. Funciones Asíncronas y Promesas 19

7.4. Manejo de Datos y Transformaciones 19

8. ENFOQUE QA / TESTEABILIDAD (WEB) **20**

9. CHECKLIST DE ENTREGA (DEFINITION OF DONE - TÉCNICA) **22**

10. EJEMPLOS DE ANTIPATRONES (SUGERENCIA) **23**

El presente documento define los estándares mínimos de codificación, criterios de entrega y lineamientos de testeabilidad que deben cumplir los entregables de desarrollo antes de su ingreso al proceso de Quality Assurance (QA). Su objetivo es reducir la complejidad, prevenir deuda técnica y asegurar una base de calidad que permita la ejecución efectiva de pruebas funcionales, técnicas y de aceptación.

1. INTRODUCCIÓN Y ALINEACIÓN

Este estándar define las **reglas de juego** técnicas para el desarrollo. La correcta aplicación de estas directrices garantiza que el producto de software entregado al equipo de Quality Assurance (QA) posea la calidad intrínseca necesaria para someterse a pruebas rigurosas.

Principio fundamental: El código debe ser una herramienta de comunicación. Debe estar listo para ser leído y comprendido por otros ingenieros (QA incluido), no solo compilado por máquinas.

2. REFERENCIAS Y RESPALDO TÉCNICO

Nuestros estándares se basan en marcos de calidad reconocidos internacionalmente, asegurando la mantenibilidad, fiabilidad y escalabilidad del producto:

ISO/IEC 25010 (Modelo de Calidad de Software)

El código y los componentes entregables deben satisfacer explícitamente los siguientes atributos:

- **Mantenibilidad:** El código debe ser comprensible y modificable. Se exige el atributo de **"Facilidad de Análisis"** (la capacidad del código para ser diagnosticado) y **"Modularidad"** (ver Glosario: Modularidad (25); la capacidad de un sistema de estar compuesto por componentes discretos).
- **Fiabilidad:** El código debe ser capaz de mantener un nivel de rendimiento especificado bajo condiciones especificadas, con un enfoque particular en la robustez del manejo de errores.

Clean Code y Arquitectura Sólida

Se exige la adhesión a los siguientes principios como base teórica de la codificación:

- **Principios SOLID:** Las clases deben tener una única responsabilidad (*Single Responsibility Principle*) y deben estar abiertas a extensión, pero cerradas a modificación (*Open-Closed Principle*).
- **Principio DRY (*Don't Repeat Yourself*):** Evitar la duplicación de lógica o de componentes de código.

2.1 Glosario de Conceptos y Definiciones Operativas

ID	Concepto	Definición
1	Antipatrón	Práctica que parece una buena solución, pero que a largo plazo genera problemas de calidad, mantenimiento o comprensión del software.
2	API REST	Forma estandarizada para que sistemas se comuniquen entre sí a través de internet mediante solicitudes y respuestas.
3	Archivo .env	Archivo local donde se almacenan configuraciones sensibles del sistema. No debe subirse al repositorio.
4	Archivo .env.example	Archivo de referencia que indica qué configuraciones necesita el sistema, sin contener valores reales.
5	Async / Await	Forma de escribir procesos que pueden tardar sin bloquear el funcionamiento general del sistema.
6	Backend	Parte del sistema que procesa la información y la lógica interna, no visible para el usuario.

ID	Concepto	Definición
7	Bajo Acoplamiento	Diseño donde las partes del sistema dependen poco entre sí, facilitando cambios y pruebas.
8	Callback	Función que se ejecuta automáticamente como respuesta a otra acción o evento.
9	CI / Integración Continua	Proceso automático que revisa y prueba el código cada vez que se realiza un cambio.
10	Cobertura de Código	Porcentaje del código que es ejecutado durante pruebas automáticas.
11	Complejidad Ciclomática	Medida que indica cuántos caminos lógicos distintos puede seguir una función.
12	Controller	Componente que recibe solicitudes y las deriva a la lógica correspondiente.
13	Deployar	Publicar o instalar una aplicación para que esté disponible para usuarios.
14	Endpoint	Dirección específica a la que se envía una solicitud para obtener una respuesta del sistema.
15	Estado	Valor actual de un dato que influye en el comportamiento del sistema.

ID	Concepto	Definición
16	Excepción	Error o situación inesperada que ocurre durante la ejecución del sistema.
17	Frontend	Parte del sistema con la que interactúa directamente el usuario.
18	Función Asíncrona	Función que se ejecuta sin detener el resto del sistema mientras espera un resultado.
19	Función con Propósito Acotado	Función diseñada para realizar una única tarea clara y específica.
20	Gate de Calidad	Punto de control donde se decide si un entregable cumple los criterios para avanzar.
21	Hardcodear	Escribir valores fijos directamente en el código en lugar de usar configuraciones externas.
22	HTTP Request	Solicitud enviada por un sistema para pedir información o ejecutar una acción.
23	Lógica de Negocio	Reglas que definen cómo debe comportarse el sistema según el dominio del problema.
24	Mock	Objeto simulado usado en pruebas para reemplazar componentes reales.

ID	Concepto	Definición
25	Modularidad	Capacidad del sistema de dividirse en partes independientes.
26	OWASP	Organización internacional que define buenas prácticas de seguridad para aplicaciones.
27	OWASP Nivel 2	Conjunto intermedio de controles de seguridad para prevenir vulnerabilidades comunes.
28	Payload	Datos enviados dentro de una solicitud a un sistema.
29	Pipeline	Secuencia automática de pasos para validar, probar y preparar el software.
30	Promise	Objeto que representa un resultado que estará disponible en el futuro.
31	Pruebas de API	Pruebas que validan el funcionamiento de interfaces de comunicación entre sistemas.
32	Pruebas Unitarias	Pruebas que validan pequeñas partes del sistema de forma aislada.
33	Render Condicionado	Técnica que decide qué mostrar en pantalla según ciertas condiciones.

ID	Concepto	Definición
34	Repositorio	Lugar central donde se almacena y versiona el código del proyecto.
35	Secreto	Dato sensible como contraseñas, claves o tokens que deben protegerse.
36	Single Responsibility Principle	Principio que indica que una parte del sistema debe tener una sola responsabilidad.
37	Stack Trace	Detalle técnico que muestra por qué ocurrió un error interno.
38	Status Code	Código numérico que indica el resultado de una solicitud a un sistema.
39	Stub	Implementación simple usada en pruebas para simular dependencias.
40	Thin Controller	Controller que contiene mínima lógica y delega el procesamiento.
41	Token	Valor digital usado para identificar o autenticar a un usuario o sistema.
42	Use Case	Descripción de una acción que un usuario puede realizar en el sistema.

ID	Concepto	Definición
43	Variable de Entorno	Valor externo al código que configura el comportamiento del sistema.
44	White-box Testing	Técnica de prueba que analiza la lógica interna del sistema.

3. REGLAS DE CODIFICACIÓN Y COMPLEJIDAD (HARD RULES)

El incumplimiento de estas reglas resultará en el rechazo inmediato del entregable y su devolución al desarrollador para corrección.

El rechazo aplica al código nuevo o modificado en el entregable (PR/commit/feature), no a código legado fuera del alcance del cambio, salvo que el cambio lo toque o lo afecte directamente.

Naming Conventions (Convenciones de Nombres)

Elemento	Regla Obligatoria	Ejemplo Aceptable	Ejemplo Prohibido
Variables/Propiedades	Descriptivas, en **inglés**. Usar camelCase.	clientAddress, requestPayload	var1, x, data, dataStructure
Métodos/Funciones	Descriptivos, en **inglés**. Deben ser **verbos** que indican una acción. Usar camelCase.	calculateTotal, fetchUserData, validateInput	calculate, total, func1

Elemento	Regla Obligatoria	Ejemplo Aceptable	Ejemplo Prohibido
Clases/Interfaces	Descriptivos, en **inglés**. Deben ser **sustantivos** o frases nominales. Usar PascalCase.	UserService, TransactionProcessor, AuthInterface	MyClass, Processor

Excepción: términos de dominio/regulatorios o nombres propios del negocio pueden mantenerse en español si son estándar del cliente o del país, siempre que sean consistentes y descriptivos.

Funciones y Complejidad Ciclomática

- **Longitud Máxima:** Las funciones o métodos deben mantenerse por debajo de las **30 líneas de código**. Se consideran líneas de código efectivas (excluye líneas en blanco). Comentarios cuentan solo si forman parte del bloque lógico (p. ej., docstrings extensos deben moverse a documentación si afectan el límite).
- **Refactorización:** El objetivo es reducir la complejidad ciclomática (ver Glosario: Complejidad Ciclomática (11)), asegurando que cada función tenga un propósito claro y acotado (ver Glosario: Función con Propósito Acotado (19)), evitando mezclar lógica de decisión, validación y procesamiento en un mismo bloque.

Regla de Seguridad – Manejo de Secretos y Configuración Sensible (HARD RULE)

Está terminantemente prohibido:

- Hardcodear credenciales, tokens, claves API, secretos o contraseñas en el código fuente.
- Versionar archivos de configuración que contengan secretos (por ejemplo, archivos .env).
- Almacenar secretos en constantes, enums, archivos de configuración o valores por defecto.
- Subir al repositorio tokens, claves o credenciales, incluso si corresponden a entornos de prueba o desarrollo.

Regla obligatoria:

Toda configuración sensible (credenciales, tokens, claves, secretos, endpoints privados) DEBE ser gestionada exclusivamente mediante variables de entorno.

3.1. Complejidad de Bucles y Control de Flujo

Se debe evitar concentrar lógica compleja en una sola línea o bloque de código, especialmente cuando involucra múltiples decisiones o validaciones encadenadas. Se prohíben las estructuras de control que dificulten la lectura y el white-box testing (ver Glosario: White-box Testing (44)):

- **Evitar bucles anidados profundos:**
  - No más de **2 niveles de anidación** (for/while/if).
  - Si se requiere mayor profundidad, se debe refactorizar en funciones auxiliares con nombres descriptivos.
- **Evitar condiciones complejas en una sola línea:**
  - Evitar encadenar múltiples operadores (&&, ||) sin paréntesis.
  - Si una condición es difícil de interpretar a simple vista, extraerla a función/variable descriptiva.
  - Usar funciones descriptivas (ej: isUserAuthorized()) en lugar de condiciones largas.
- **Preferir retornos tempranos (*early return*):**
  - Reducir bloques if/else extensos que generen múltiples niveles de anidamiento, privilegiando retornos tempranos que simplifiquen el flujo y faciliten el seguimiento del camino lógico principal.
  - Mejorar legibilidad y trazabilidad de errores, ya que el camino "feliz" queda menos anidado.
- **Evitar lógica oculta en *callbacks* (ver Glosario: Callback (8)):**

- Evita escribir cálculos complejos o reglas dentro de funciones como map, filter o reduce; en su lugar, extrae esa lógica a una función independiente con un nombre que explique qué hace. Esto permite que el código se lea como una frase clara en lugar de una fórmula difícil de descifrar; por ejemplo, es mucho más entendible usar productos.map(calcularIVA) que escribir toda la operación matemática directamente dentro del paréntesis.
- Extraer la lógica a funciones con nombre, Ej: calcularIVA.

3.2. Complejidad Ciclomática (Regla Explícita)

La complejidad ciclomática (ver Glosario: Complejidad Ciclomática (11)) corresponde a la cantidad de caminos lógicos posibles dentro de una función y debe mantenerse controlada para facilitar la comprensión, el testeo y el mantenimiento del código. Toda función debe mantener una complejidad ciclomática intrínsecamente baja.

- **Indicadores de alerta:**
  - Múltiples if/else encadenados.
  - Se prohíbe el uso de bloques switch que ejecuten lógica de cálculos o transformaciones extensas dentro de sus casos (case). El switch debe actuar exclusivamente como un selector que delega la ejecución a funciones o métodos independientes y descriptivos, garantizando que cada rama de decisión sea atómica, fácil de testear y no incremente artificialmente la complejidad ciclomática de la función principal.
  - Uso excesivo de *flags* booleanos para controlar el flujo dentro de la función.
- **Criterio práctico:** Si una función concentra múltiples decisiones, validaciones o flujos alternativos que dificultan su lectura directa, debe refactorizarse en unidades más simples y específicas.

Manejo de Excepciones

- **Prohibición de Silencio:** Está terminantemente prohibido el **"swallowing exceptions"** (ej: bloques catch vacíos o que solo registran un *log* sin acción correctiva/informativa).
- **Propagación y Registro:** Las excepciones deben ser capturadas, registradas adecuadamente (nivel ERROR, WARN, etc.) y, si es necesario, relanzadas o transformadas en una excepción más clara para el nivel superior.

4. PROTOCOLO DE COMENTARIOS (DOCSTRING)

El propósito del comentario es mejorar la Facilidad de Análisis del código, especialmente en la fase de Pruebas de Caja Blanca por parte de QA.

Comentarios Generales

- **Evitar lo Obvio: No comentar lo que el código ya explica por sí mismo** (ej: i = i + 1 // incrementa i).
- **OBLIGATORIO: Comentar el "POR QUÉ":** El comentario debe explicar la **lógica compleja, las decisiones de diseño atípicas o los *workarounds*** que se implementaron.

Encabezados de Función (Docstrings)

Todo método o función pública debe incluir un encabezado estandarizado para facilitar la revisión y el diseño de pruebas:

- **Descripción:** Un resumen conciso de lo que hace la función.
- **Parámetros (*Params*):** Tipo y descripción de cada parámetro de entrada.
- **Retorno (*Return*):** Tipo y descripción del valor de salida.
- **Excepciones (*Throws*):** Mención de las excepciones conocidas que el método puede lanzar.

Ejemplo de docstring de función con descripción, parámetros, retorno y manejo de errores:

```
/**
 * Processes a purchase order payment by validating the available balance
 * and updating the transaction status in the database.
 *
 * @param {string} orderId - The unique identifier for the order to be processed.
 * @param {number} amount - The total amount to be debited (must be greater than
zero).
 * @param {object} paymentGateway - Configured instance of the payment gateway
service.
 * * @returns {Promise<{transactionId: string, status: string}>} An object
containing the transaction ID and final status.
 * * @throws {InsufficientFundsError} If the customer has insufficient funds.
 * @throws {InvalidAmountError} If the provided amount is zero or negative.
 * @throws {GatewayTimeoutError} If the payment gateway service times out.
 */
async function processOrderPayment(orderId, amount, paymentGateway) {
  // ... function logic ...
}
```

Nota sobre metadatos de autoría:

No se requiere ni se recomienda incluir información de autor, fecha o nombre del proyecto dentro del código fuente, dado que estos datos deben obtenerse desde el sistema de control de versiones (repositorio, commits e historial).

Comentarios de contexto en puntos de entrada:

En archivos que representen puntos de entrada relevantes del sistema (por ejemplo, inicialización de la aplicación, configuración principal o bootstrap), se recomienda incluir un comentario de contexto que describa brevemente:

- El propósito del archivo o módulo.
- Su responsabilidad dentro del sistema.
- Supuestos importantes o decisiones de diseño no evidentes.

Estos comentarios deben enfocarse en el "qué" y el "por qué", no en la autoría.

Ejemplo de comentario de contexto que define el propósito y la responsabilidad de un módulo o punto de entrada del sistema:

```
/**
 * @file Main application entry point.
 * * @description
 * This module serves as the primary orchestrator for the application startup.
 * * Responsibilities:
 * - Initialize core configuration and environment variables.
 * - Register and manage global dependencies.
 * - Bootstrap essential core services.
 * * Out of Scope:
 * - Business logic or domain rule implementation.
 * - Direct database access or data manipulation.
 */
```

5. ESTÁNDARES DE LENGUAJE Y DOCUMENTACIÓN DE PROYECTO

El lenguaje utilizado en la documentación debe alinearse con el propósito de la audiencia para maximizar la claridad, evitar ambigüedades y mantener la consistencia con las convenciones de nombres de código (Sección 3).

Documentación Técnica (Para el Equipo de Desarrollo)

- **README Principal (Repositorio):** Se recomienda un enfoque **bilingüe**. El archivo README.md principal debe mantenerse en inglés (estándar internacional), con un enlace prominente a una versión en español (README.es.md).
- **Documentación de APIs (Swagger/Postman/etc.):** Debe ser escrita en **Inglés**. Esto evita la mezcla confusa de lenguajes al documentar endpoints y variables que ya están definidas en inglés en el código (ej. explicar el campo merchant_id como "Identificador del comerciante").
- **Documentación de Decisiones de Arquitectura:** Debe ser escrita en **Español** para facilitar el entendimiento y las discusiones dentro del equipo local.
- **Comentarios en Código y Nombres de Funciones/Variables:** Véase la Sección 3 (Naming Conventions).

Documentación de Proyecto y Usuario

- **Requisitos, Historias de Usuario (Jira) y Reglas de Proyecto:** Deben ser escritos en **Español** para garantizar la comprensión clara y sin ambigüedades por parte de los stakeholders, Product Owners y el equipo de QA.
- **Manuales de Usuario y Guías de Operación (Externos):** Deben ser escritos en Español..

5.1 Estándar de Lenguaje para Bases de Datos

- **Esquema de Base de Datos y Objetos (HARD RULE):** Todos los nombres del esquema de la base de datos (tablas, columnas, procedimientos almacenados, vistas, etc.) DEBEN estar en Inglés. Esto asegura la consistencia con las convenciones de nombres del código fuente (Sección 3: Naming Conventions) y la documentación de APIs.
- **Convención de Nombres:** Se debe utilizar **snake_case** (nombre_del_objeto) para todos los nombres de objetos de la base de datos (ej. user_accounts, request_log).

6. SEGURIDAD DE APLICACIONES (OWASP NIVEL 2)

Esta sección define los **controles mínimos obligatorios de seguridad** que deben implementarse **desde la construcción del código**, alineados con el estándar **OWASP – Nivel 2**, con el objetivo de **mitigar vulnerabilidades críticas antes del ingreso a QA**.

El incumplimiento de cualquiera de los puntos de esta sección **habilita el rechazo inmediato del entregable** en el gate de calidad.

6.1 Principios Generales de Seguridad (Shift-Left Security)

- La seguridad **no se valida al final**, se **construye desde el código**.
- Todo código nuevo o modificado debe asumir un **modelo de amenaza básico**.
- QA evaluará seguridad **desde revisión estática + pruebas funcionales**, no solo en producción.

6.2 Controles OWASP Nivel 2

Autenticación y Autorización

- Prohibido implementar autenticación “custom” sin librerías estándar.

- Toda acción sensible **debe validar permisos explícitamente**.
- Prohibido confiar en datos del frontend para autorización.

Validación y Sanitización de Entradas

- **Toda entrada externa** (HTTP, headers, params, body, env) debe validarse.
- Prohibido confiar en validaciones solo del frontend.
- Usar validaciones explícitas por tipo, rango y formato.

Manejo Seguro de Errores

- **Prohibido exponer:**
  - Stack traces
  - Mensajes internos
  - Rutas, secretos o configuraciones
- **Los errores deben ser:**
  - Técnicos hacia logs
  - Controlados hacia el usuario
- **Lenguaje de Logs Técnicos:** Todo mensaje de log interno **(ERROR, WARN, INFO, DEBUG)** y las trazas técnicas **DEBEN** ser escritos en **Inglés** para mantener la consistencia con el código fuente y facilitar el uso de herramientas de monitoreo internacional.

Dependencias

- Prohibido usar librerías sin mantenimiento o con vulnerabilidades críticas conocidas.
- Se recomienda escaneo básico de dependencias (npm audit, etc.).

Configuración Segura

- Prohibido hardcodear:
  - Credenciales
  - Tokens
  - Claves API
- Toda configuración sensible debe provenir de **variables de entorno**.

6.3 Gestión de Variables de Entorno

Prohibiciones explícitas

Está **terminantemente prohibido**:

- Versionar archivos .env
- Almacenar secretos en:
    - Código fuente
    - Constantes
    - Archivos de configuración versionados
- Subir tokens, claves o credenciales al repositorio, **aunque sean de prueba**

Práctica obligatoria

El proyecto **DEBE incluir** un archivo: .env.example

- Este archivo debe contener **todas las variables requeridas**, sin valores reales:
  Ejemplo:

```
DB_HOST=

DB_PORT=

DB_USER=

DB_PASSWORD=

JWT_SECRET=

API_KEY=
```

Documentación mínima

- El README o documentación técnica debe indicar:
    - Qué variables son obligatorias
    - Para qué se usan
    - Qué pasa si no están definidas

7. ADAPTACIÓN EXPLÍCITA A PRODUCTOS WEB

Esta sección establece directrices específicas para el desarrollo de soluciones web, incluyendo *backends* (APIs) y *frontends*.

7.1. Reglas Específicas para Backend / API

- **Controladores deben ser livianos (Thin Controllers, ver Glosario: Thin Controller (40)):**
  - No incluir lógica compleja en *controllers* (ver Glosario: Controller (12)) o *endpoints* (ver Glosario: Endpoint (14)).
  - Delegar inmediatamente la lógica a *servicios* o *use-cases* dedicados (ver Glosario: Use Case (42)).
- **Validaciones:**
  - Centralizar las validaciones de datos (no repetirlas en múltiples *endpoints*).
  - Prohibido validar y procesar la lógica en el mismo método extenso.
- **Manejo de errores:**
  - Usar errores semánticos y status codes estándar de protocolo (ver Glosario: Status Code (38)) (400, 401, 403, 404, 500) según el contexto del error.
  - Prohibido devolver mensajes de error genéricos al consumidor sin contexto útil (código/causa/acción esperada) acorde al entorno.

7.1.1 Consistencia de Idioma en Endpoints (URLs)

- **Regla Obligatoria:** Las rutas o endpoints de la API (la porción de la URL que identifica el recurso) DEBEN estar en **Inglés**. Esto mantiene la alineación con las convenciones de nombrado en código y evita la mezcla de idiomas en las interfaces técnicas.
- **Convención:** Se recomienda minúsculas y el uso de **kebab-case** (ej. /user-data) o **snake_case** (ej. /user_data) para los segmentos de la ruta, siendo consistente en la elección.

7.2. Reglas Específicas para Frontend

- **Componentes con una sola responsabilidad:**
  - Separar claramente la lógica de procesamiento y decisión de la capa de presentación, evitando que una misma unidad maneje obtención de datos, control de estado y renderizado complejo simultáneamente.

    - Evitar componentes "todo en uno" que manejan *fetch* de datos, estado global y renderizado complejo.
- **Render condicionado (ver Glosario: Render Condicionado (33)):**
    - La capa de renderizado o presentación debe enfocarse principalmente en mostrar información. Evitar incluir múltiples decisiones condicionales o cálculos complejos directamente en ella; extraer esa lógica a funciones o variables descriptivas.
- **Estados:**
    - Evitar estados (ver Glosario: Estado (15)) o variables que dupliquen información existente o que puedan derivarse directamente a partir de otros datos.
    - Debe existir un único punto responsable de la actualización de cada dato, a partir del cual el resto del código consuma o derive su valor.

7.3. Funciones Asíncronas y Promesas

Se establecen reglas para el manejo de asincronía y Promises para evitar la complejidad y los *race conditions*.

- Prohibido encadenar múltiples .then() / .catch() complejos.
- **Preferir async/await** (ver Glosario: Async / Await (5)) con bloques try/catch claros para mejorar la legibilidad y la depuración del flujo de errores.
- Una función asíncrona (ver Glosario: Función Asíncrona (18)) no debe incluir las siguientes tareas en el mismo método:
    - Llamar a múltiples APIs externas.
    - Manejar lógica compleja.
    - Transformar grandes volúmenes de datos.
    - Cualquier combinación de las anteriores debe dividirse en funciones dedicadas y testeables.

7.4. Manejo de Datos y Transformaciones

- **Transformaciones de datos complejas deben:**
    - Vivir en funciones dedicadas.
    - Tener un nombre semántico claro (ej: mapApiToDomainUser).
    - Ser testeables de forma aislada.
- **Evitar lógica "en línea":**

- No transformar objetos grandes directamente en el *return* o como parte de la definición de una variable sin una función intermedia.

8. ENFOQUE QA / TESTEABILIDAD (WEB)

El equipo de QA requerirá acceso al código y a las pruebas unitarias del desarrollador para validar la lógica interna del sistema. Se refuerza el vínculo entre el diseño de código y la facilidad de prueba.

Entrega de Pruebas Automatizadas (Base)

Dado el stack tecnológico definido para los proyectos, el entregable debe incluir una base de pruebas automatizadas que valide el comportamiento del código nuevo o modificado.

Estas pruebas se clasifican en:

a) Pruebas Unitarias (requeridas, ver Glosario: Pruebas Unitarias (32)): Validan funciones, métodos o módulos de forma aislada, sin depender de infraestructura externa (red, base de datos real, servicios de terceros), utilizando mocks o stubs (ver Glosario: Mock (24); Glosario: Stub (39)) cuando corresponda.

b) Pruebas de API / HTTP (complementarias, ver Glosario: Pruebas de API (31)): Validan endpoints REST (ver Glosario: API REST (2)) mediante requests HTTP (por ejemplo, envío de JSON y validación de respuestas). Estas pruebas permiten verificar conectividad e integración básica, pero **no reemplazan** las pruebas unitarias cuando exista lógica relevante en el código.

El objetivo de esta base es asegurar que el código sea testeable, comprensible y extensible por el equipo de QA.

Herramientas y formato de pruebas:
No se exige una herramienta específica para la creación de pruebas (por ejemplo, frameworks de testing, RestClient, cURL u otros).

Sin embargo, todas las pruebas automatizadas entregadas deben cumplir con lo siguiente:

- Estar **versionadas en el repositorio** del proyecto.
- Ser **reproducibles** por terceros.
- Incluir **instrucciones claras de ejecución** (README o documentación equivalente).

En el caso de pruebas HTTP, se deben adjuntar los archivos necesarios (colecciones, scripts o comandos) para su ejecución directa.

Cobertura de Código Mínima

Se establece una **Cobertura de Código Mínima del 70% (líneas, ramas)** como requisito de base de las **pruebas unitarias entregadas por el equipo de desarrollo**, previo al ingreso del código al entorno de QA.

Esta cobertura representa un **nivel mínimo de validación inicial**, cuyo propósito es asegurar que el código no sea una "caja negra" y que exista una base sobre la cual el equipo de QA pueda **revisar, extender, modificar o complejizar** las pruebas según los criterios de calidad definidos.

La cobertura del 70% **no corresponde a la cobertura final del proyecto**, ni reemplaza las actividades de prueba funcional, integración o aceptación realizadas por QA.

La medición de la cobertura debe provenir del reporte del pipeline/CI (ver Glosario: Pipeline (29)) o herramienta acordada por el equipo, considerando únicamente el código del entregable y excluyendo archivos generados automáticamente.

La responsabilidad de alcanzar esta cobertura mínima inicial recae en el equipo de desarrollo. La evolución y ampliación de las pruebas forma parte del proceso de aseguramiento de calidad liderado por QA.

Código Testeable

El diseño de la aplicación debe facilitar activamente la ejecución de pruebas, permitiendo aislar unidades de código, controlar dependencias y validar comportamientos de forma predecible.

- **Bajo Acoplamiento (ver Glosario: Bajo Acoplamiento (7)):** Evitar el acoplamiento fuerte entre módulos, clases o funciones. Esto es esencial para que QA pueda aislar y probar unidades específicas.
- **Refuerzo de la Testeabilidad (Web):**
    - Todo código debe ser: **Legible**, **Predecible** y **Fácil de aislar** para pruebas (por ejemplo, mediante técnicas de aislamiento de dependencias).

- **Señales de código NO testeable:** Dependencias creadas internamente (en lugar de inyectadas), uso directo de variables globales o lógica dependiente del entorno sin abstracción.

9. CHECKLIST DE ENTREGA (DEFINITION OF DONE - TÉCNICA)

Antes de mover cualquier ítem a la columna **"QA Ready"**, el desarrollador debe verificar y garantizar el cumplimiento de los siguientes puntos.

Requisito de Entrega	**Cumplido (Sí/No)**
1. Código adhiere a Naming Conventions (punto 3).	
2. Funciones refactorizadas y ninguna excede el límite de 30 líneas (punto 3).	
3. Todos los métodos públicos tienen un Docstring con (Params, Return, Throws) (punto 4).	
4. Se han verificado las reglas de Manejo de Excepciones (prohibido catch vacío) (punto 3).	
5. Se evitaron anidamientos de control de flujo > 2 niveles (punto 3.1).	
6. Funciones asíncronas utilizan async/await preferentemente (punto 6.3).	
7. Funciones cumplen propósito acotado y complejidad controlada (puntos 3, 3.1 y 3.2).	
8. Los *controllers* (Backend) son livianos, delegando a servicios o *use-cases* (punto 6.1).	

Requisito de Entrega	Cumplido (Sí/No)
9. Las pruebas unitarias entregadas por desarrollo alcanzan una Cobertura de Código ≥ 70% (líneas y ramas), según reporte de CI (punto 6).	
10. No existen secretos hardcodeados en el repositorio.	
11. Toda configuración sensible utiliza variables de entorno.	
12. El proyecto incluye archivo .env.example sin valores reales.	
13. No se versionan archivos .env ni configuraciones con secretos.	
14. Los errores no exponen información sensible.	
15. Se cumplen los controles de Seguridad OWASP Nivel 2 (Sección 5).	

10. EJEMPLOS DE ANTIPATRONES (SUGERENCIA)

Nota: *Esta sección es de carácter referencial y educativo. No condiciona por sí sola la aceptación del entregable, pero sirve como guía práctica de los patrones esperados y prohibidos.*

Esta sección tiene como objetivo servir como referencia práctica para identificar patrones prohibidos y su refactorización esperada, facilitando la alineación entre desarrollo y QA.

- **Función Larga vs. Funciones Pequeñas:** Demostración de cómo refactorizar una función de 60 líneas en varias funciones de 10-15 líneas.

- **Condición Compleja vs. Función Descriptiva:** Mostrar el reemplazo de una condición booleana extensa por una función con nombre semántico.
- **Componente Sobrecargado (*Frontend*) vs. Separación Clara:** Ejemplo de cómo dividir un componente monolítico en componentes de presentación y contenedores de lógica.